

QUDA MultiGrid 的算法理解与 PyQCU 复现

P/R 聚合、Galerkin 粗化、Clover/Gauge、Schur 奇偶与递归预条件

代码审计与代数验证

PyQCU · Lattice QCD

2026 年 8 月 29 日

结论先行: QUDA 的代数主线已落到 PyQCU 参考实现

已完成

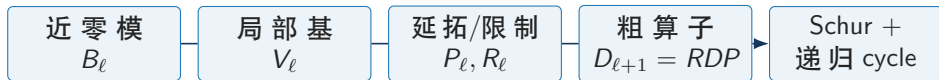
新增独立的 QudaTransfer、QudaCoarseOperator、ParitySchurOperator 与 QudaMultigrid; 旧 multigrid 原样保留。

直接证据

CPU、4⁴、复数单精度回归 **PASS: 21 passed**。新增 setup 语义、solver 家族、伴随与缺失伴随错误路径; 既有 *RP*、Galerkin、Schur 和 full residual 仍通过。

边界

这是可验证的 Python/QCU 复现路径, 不是 QUDA 生产 GPU kernel。Python setup 已覆盖 null/test、CG/CA-CG/Krylov; compact Schur setup、QCU 生产 setup 与硬件性能仍 **未验证**。



阶段判断: QUDA 风格 Python MultiGrid 的层级代数与 null-vector setup 已形成可验证实现; QCU 仍已通过 33 点 coarse halo 和 2/4 rank Clover MG 小格闭环; 下一步是 compact Schur/生产 setup 与大格性能对照。

来源: 实现: pyqcu/solver/_quda_multigrid.py:218-1689; 测试: pyqcu/testing/python/test_quda_multigrid.py:76-389; 本次命令: pytest -q

pyqcu/testing/python.
代码审计与代数验证

QUDA MG 复现

2/21

第一性原理: smoother 消高频, 粗空间承接长程误差

误差分解与 Galerkin 投影

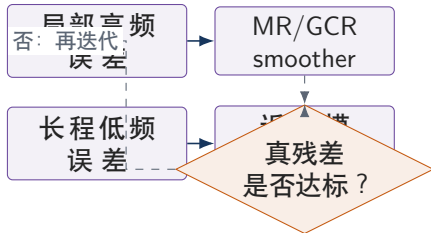
设 $e = e_{\text{high}} + e_{\text{low}}$ 。局部平滑器快速削弱 e_{high} , 而慢变的 e_{low} 由近零模张成的粗空间表示:

$$\begin{aligned} e_{\text{low}} &\simeq P e_c, & e_c &\simeq (RDP)^{-1} R r, \\ D_c &= RDP, & R &= P^\dagger. \end{aligned}$$

不变量

$P^\dagger P = I_c$ (局部 aggregate 意义); D_c 的 native apply 必须与显式 $P \rightarrow D \rightarrow R$ 相同。

来源: QUDA 算法入口: [refer/git-rep/quda/lib/multigrid.cpp](https://github.com/quda/lib/multigrid.cpp):72-197,1131-1224; Galerkin/验证: [refer/git-rep/quda/lib/coarse_op.cuh](https://github.com/quda/lib/coarse_op.cuh):1035-1459, [refer/git-rep/quda/lib/multigrid.cpp](https://github.com/quda/lib/multigrid.cpp):783-1069。



物理检查

规范场 U_μ 、Clover 局部项 C 和 hopping 共同定义 D ; 不能只把 hopping 粗化而丢掉 local term, 否则 Schur 与粗层物理算子不一致。

层级 setup: 每层都是 Transfer + coarse Dirac + 两侧平滑 + 粗解器

输入: $(D_\ell, B_\ell, b_\ell, \text{geometry}, \text{precision})$

1. map: $a(x) = \lfloor x/b \rfloor$, 建立 fine/coarse 索引

2. transfer: $B_\ell \xrightarrow{\text{aggregate CGS}} V_\ell \rightarrow (P_\ell, R_\ell)$

3. coarse: $D_{\ell+1} = R_\ell D_\ell P_\ell \rightarrow (X_\ell, Y_\ell)$

4. PC: $X_\ell^{-1}, Y_{\text{hat}_\ell} \rightarrow \text{even/odd Schur}$

5. recurse: $B_{\ell+1}$ 与下一层 $D_{\ell+1}$; 底层接 Krylov

输出: pre/post smoother + coarse solver + level state

QUDA 的构造顺序



PyQCU 对应

`setup()` 建立 levels/transfers;
`cycle()` 返回校正。

来源: QUDA: `multigrid.cpp:72-434,564-702`; PyQCU: `_quda_multigrid.py:986-1211`; 图为控制流抽象。

P/R 的核心是同一套几何 map 与 spin/parity map

几何与自旋映射

$$a(x) = \left(\left\lfloor \frac{x_0}{b_0} \right\rfloor, \dots, \left\lfloor \frac{x_3}{b_3} \right\rfloor \right),$$
$$q(s, p) = \begin{cases} \lfloor s/b_s \rfloor, & \text{Wilson/Clover,} \\ p(x), & \text{staggered } (b_s = 0). \end{cases}$$

Wilson/Clover 典型为 $4 \rightarrow 2$ spin、 $b_s = 2$;
staggered 为 $1 \rightarrow 2$, 粗 spin 承载 checkerboard parity。

维数

$D_f = 4 \times 3 = 12$; 粗层 $D_c = 2 \times N_{\text{vec}}$ 。
dof_list=[12,24,24] 解释为每层总 DOF, 而不是每个 coarse-spin block 的向量数。

来源: QUDA: transfer.cpp:200-328; PyQCU: _quda_multigrid.py:197-460。

代码审计与代数验证

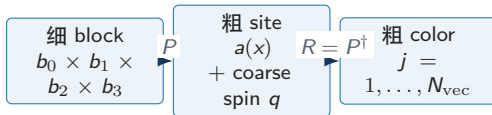
P 与 R

对每个 aggregate a 和 coarse spin q , $V_{s,c,q,j}(x)$ 是局部正交列:

$$(P\eta)_{sc}(x) = \sum_j V_{scqj}(x) \eta_{qj}(a(x)),$$

$$(R\psi)_{qj}(a) = \sum_{x \in a, s, c} V_{scqj}^*(x) \psi_{sc}(x).$$

因而 $R = P^\dagger$; $RP\eta \simeq \eta$ 是最先应测的不变量。



QUDA MG 复现

5/21

块 CGS: 正交对象是 aggregate-local 的列, 而不是全局向量

for each aggregate a and coarse spin q :

load $v_j \leftarrow B_j$ (首轮) 或 V_j (后续轮)

$h_{ij} = \langle v_i, v_j \rangle_{a,q}, \quad v_j \leftarrow v_j - h_{ij} v_i$

$v_j \leftarrow v_j / \|v_j\|_{a,q}$ (逐列归一化)

$j = 0, \dots, N_{\text{vec}} - 1$

repeat $n_{\text{block_ortho}}$ passes

return V ; 再以同一 V 定义 P, R

容量约束

$N_{\text{vec}} \leq N_c |a| b_s$ (Wilson/Clover); staggered 有效容量为 $N_c |a| / 2$ 。超出局部维数时只能得到零列/相关列。

源码对应

QUDA kernel 按 aggregate、parity、chirality 和 vector tile 并行; 首轮从 B 读, 之后从 V 读, 最后归一化保存。

PyQCU 复现

`_cg_orthogonalise()` 使用复数 CGS, 逐 block 批量执行;
`_block_orthogonalise()` 分别处理 Wilson/Clover 与 staggered slow path。

验证

实测 block-ortho 最大 Gram 误差:
 5.34×10^{-7} (多层例), staggered:
 3.38×10^{-7} 。

来源: QUDA: `block_orthogonalize.cuh:149-239, transfer.cpp:152-163`; PyQCU: `_quda_multigrid.py:155-185, 285-351`。

粗算子保留 local Clover 与 Gauge hopping: $D_c = RDP$

动作形式

$$D_c \eta(q) = X(q) \eta(q) + \sum_{\mu=0}^3 [Y_{\mu}^{+}(q) \eta(q + \hat{\mu}) + Y_{\mu}^{-}(q) \eta(q - \hat{\mu})].$$

X 是目标点 local block; $Y^{\pm}$ 是 coarse link。周期边界与邻居坐标必须和 ...xyz 布局一致。

物理来源

细格 $D(U, C, \kappa)$ 经 $V^{\dagger} D V$ 投影: hopping 贡献 $V^{\dagger} U_{\mu} V$, Clover/local 贡献 $V^{\dagger} C V$; 无 Clover 时保留 identity/local diagonal。

来源: QUDA: coarse_op.cuh:1040-1459, dirac_coarse.cpp:121-239; PyQCU: _quda_multigrid.py:573-805。

对象	QUDA 与 PyQCU 的对应
QUDA Y	UV 后再 VUV, 按 forward/backward、dimension 累加, 必要时双向 link 与 halo
QUDA X	coarse Clover/local field; coarse-on-coarse 仍按 $V^{\dagger} C V$ 组织
PyQCU materialize	逐粗自由度 probe: $P \rightarrow D \rightarrow R$, 按 displacement 聚成 block
PyQCU lazy	每次直接执行 $R(D(P\eta))$; 避免大格式矩阵

边界修正

粗 extent 为 2 时 $+\hat{\mu}$ 与 $-\hat{\mu}$ 是同一邻点; 实现将两份方向系数各折半, 避免 apply 重复计数。

Clover、Gauge 和粗层算子：接口不同，代数对象相同

PyQCU 细层入口

$U[3, 3, 4, X, Y, Z, T]$
 $C[4, 3, 4, 3, X, Y, Z, T]$ (可选) 无 Clover 时细
 $\downarrow$ `dslash.operator(U, C, kappa)`
 $D_f[12, X, Y, Z, T]$, $X_f = I$ 或 M_C
层 diagonal 是 identity; 有 Clover 时取
`operator.sitting.M`。这使 parity Schur 总能调用局部逆。

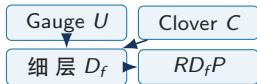
粗层递归

第一层得到 $D_1 = R_0 D_0 P_0$; 下一层把 D_1 作为新的 fine operator, 再用 B_1 、 P_1/R_1 继续粗化。

来源: PyQCU: `_quda_multigrid.py:1003-1116`; QUDA: `dirac_coarse.cpp:121-626`。

QUDA 细节

`createY()`: 粗 link Y 与 local X ;
`createCoarseOp()`: aggregate-only 粗化;
`DiracCoarsePC`: 以 $Y_{hat} + X$ 施加 PC hopping;
`coarse-on-coarse`: C 进入下一次 VUV/local。



当前实现级别

确证 代数检查与 QCU 小格通信通过; **未验证**
QUDA 生产侧 UV/VUV、field order、
NVSHMEM 与全面性能等价。

预条件粗算子：先消去 even，再在 odd Schur 上求解

块矩阵与 Schur 补

$$D = \begin{pmatrix} X_e & D_{eo} \\ D_{oe} & X_o \end{pmatrix}, \quad S_o = X_o - D_{oe}X_e^{-1}D_{eo},$$

$$b_o^S = b_o - D_{oe}X_e^{-1}b_e, \\ x_e = X_e^{-1}(b_e - D_{eo}x_o).$$

这里 odd 是被保留的目标 parity；交换目标 parity 只改变选取，不改变公式。

QUDA 的 X^{-1} 与 $Yhat$

$$Yhat_{\mu}^{+}(q) = X^{-1}(q)Y_{\mu}^{+}(q);$$

backward link 存在 $q - \hat{\mu}$ ，并携带相应的共轭转置/右侧 $X^{-\dagger}$ 。

DiracCoarsePC::Dslash 只施加 PC hopping；
M() 与 prepare/reconstruct() 负责 full/PC 转换。

PyQCU 明确的语义

$$\mathcal{A}_{pc}(x) = X^{-1}(D_c - X)x,$$

$$\mathcal{A}_{full}(x) = X^{-1}D_c x = x + \mathcal{A}_{pc}(x).$$

对应代码：preconditioned_apply、
preconditioned_full_apply。

来源：QUDA: coarse_op_preconditioned.cuh:48-120、dirac_coarse.cpp:506-626；PyQCU: _quda_multigrid.py:804-872,908-983。

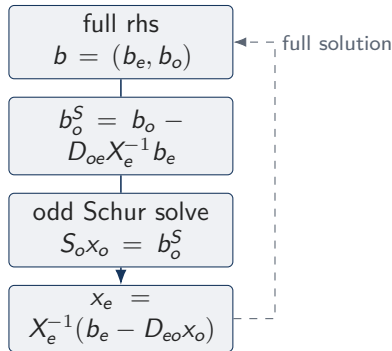
奇偶处理从 finest 延伸到每一个 coarse level

QUDA 的运行期选择

`MG::operator()` 根据 `matpc_type` 选择 even/odd;
检查 outer/inner solution type 是否一致。若 coarse 是 MATPC, smoother 必须是 preconditioned solver;
不一致时必须 reconstruct 后重新计算 residual。

PyQCU 的新路径

Checkerboard 保存两 parity 的确定性 index;
`ParitySchurOperator` 对任意层的 full coarse operator 构造 compact Schur、rhs、reconstruct 和 MR correction。



实现约束

`use_parity=True` 要求每层已构造 X/Y ;
大格矩阵自由模式关闭 `parity`, 避免隐式 `materialize`。

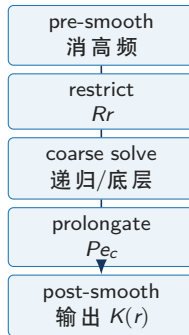
来源: QUDA: `multigrid.cpp`:1131-1221; PyQCU: `_quda_multigrid.py`:874-983,1205-1207,1254-1281。

V-cycle 与外层 FGMRES: 粗解是可变化的右预条件器

$x_\ell \leftarrow 0$ 或给定初值;
 $x_\ell \leftarrow S_{\text{pre}}(b_\ell, x_\ell);$
 $r_\ell \leftarrow b_\ell - D_\ell x_\ell;$
 $r_{\ell+1} \leftarrow R_\ell r_\ell;$
 $e_{\ell+1} \leftarrow \text{recursive coarse solve}(r_{\ell+1});$
 $x_\ell \leftarrow x_\ell + P_\ell e_{\ell+1};$
 $x_\ell \leftarrow S_{\text{post}}(b_\ell, x_\ell);$
return $K(b_\ell) = x_\ell$ 。

外层

`fgmres(b,D,precond=V-cycle)` 以右预条件形式构造 Arnoldi、复 Givens、上三角回代与 restart; 每次 V-cycle 可因残差、层次或粗解精度而变化。



两种底层

小格可对显式 coarse matrix 直接 solve; 一般情形使用 coarse FGMRES。代码对 zero RHS、zero residual 和 singular breakdown 保持有限值。

来源: QUDA: `multigrid.cpp:1131-1224,564-702`; PyQCU: `_quda_multigrid.py:1254-1327`、`_gmres.py:49-156`。

新旧实现并存：新路径补全代数层，不破坏旧 API

维度	旧 solver.multigrid	新 solver.QudaMultigrid
粗自由度 transfer	flat E ; 接口历史兼容 finest/MG-0 有 parity 处理	$2 \times N_{\text{vec}}$, 显式 coarse spin 每层共享 map、 V 、 P 、 $R = P^\dagger$; 含 staggered $1 \rightarrow 2$
coarse operator	33-point/窗口化路径, 粗层不是完整 dslash	$D_c = RDP$; materialized block 或 matrix-free composition
Clover/Gauge	依赖旧粗层约定	细层复用 Wilson/Clover dslash; local X 进入每层 Schur
parity	主要在最细层	ParitySchurOperator 可作用于任意粗层
角色	历史生产/实验路径, 保留	QUDA 语义的 CPU/CUDA tensor reference

兼容策略

solver/__init__.py 同时导出旧/新类; 测试显式断言二者不是同一类, 避免补充实现覆盖旧路径。

来源: 旧: solver/_multigrid.py; 新: solver/_quda_multigrid.py:986-1407; 导出/测试: solver/__init__.py:1-12、test_quda_multigrid.py:56-58。

代数验证结果：核心恒等式与求解 residual 均通过

指标 (4^4 , c64)	实测
多层 $\ RP_\eta - \eta\ /\ \eta\ $	1.54×10^{-7}
多层 transfer 伴随相对误差	5.43×10^{-9}
多层 block Gram 最大误差	5.34×10^{-7}
多层 $\ D_c\eta - RD(P_\eta)\ /\ RD(P_\eta)\ $	7.62×10^{-8}
Schur 偶方程重构范数	3.18×10^{-7}

求解/边界	实测
多层 full residual	1.04×10^{-7}
matrix-free residual	4.07×10^{-8}
staggered RP	1.41×10^{-7}
staggered 伴随误差	9.50×10^{-9}
Wilson Schur even residual	1.14×10^{-6}
Clover Schur even residual	2.04×10^{-6}

回归闸门

21 passed in 8.45s

覆盖 transfer、Galerkin、 $X^{-1}/Yhat$ 、Schur、多层/单层、lazy、Wilson/Clover+Gauge、staggered、QCU asset、setup solver 与 FGMRES breakdown。

证据等级

以上数值是本次 CPU 小格命令的直接输出；它们证明参考实现的代数约定，不外推 CUDA kernel 的吞吐、MPI scaling 或大格迭代数。

来源：命令（在 pyqcu/testing/python）：`pytest -q test_quda_multigrid.py`；复核：`QudaMultigrid.diagnostics()`；测试：`test_quda_multigrid.py:76-389`。

关键坑点已变成可回归的不变量，而不是隐含约定

发现的边界	修复	物理/代数后果
coarse extent = 2 时 +/- 邻居相同	两方向 block 各折半	周期邻居不被 apply 两次
staggered 延拓把 coarse 坐标误当 fine 展开索引	显式按 fine site parity 选 q	1→2 spin/parity 与 P/R 一致
backward Yhat 的存储位置/右乘错误	移到 $q - \hat{\mu}$, 使用正确 $X^{-\dagger}$	PC hopping 与 QUDA link convention 对齐
mixed dtype/device	apply 前转到 V 所在 dtype/device, 再转回输入	CPU/CUDA tensor 接口不因类型漂移失效
显式 coarse 矩阵内存无上限	增加元素上限与 matrix-free 模式	大格不因 probe 直接失控; 代数仍可 lazy 验证
zero RHS / singular Arnoldi	FGMRES 提前返回、回代零 pivot 不除零	V-cycle 的零粗 residual 不产生 NaN

工程结论

这些问题来自周期索引、复数伴随、存储位置和奇偶块结构；先固定不变量，再谈 kernel 和性能。

来源：修复：solver/_quda_multigrid.py:73-88,377-495,657-862、solver/_gmres.py:31-45,77-147；QUDA：coarse_op_preconditioned.cuh:48-120。

可复现入口与当前限制：reference 已验证，QCU 已有小格闭环

最小使用形态

```
1 from pyqcu.solver import QudaMultigrid
2 mg = QudaMultigrid(
3     U=U, clover_term=C, kappa=kappa,
4     null_vectors=[B0, B1],
5     block_size=[(2,2,2,2), (1,1,1,1)],
6     max_level=3, use_parity=True)
7 x = mg.solve(b)
8
```

自定义算子则提供 fine_matvec、fine_diagonal
(启用 parity 时必需) 和可选 fine_adjoint。

已验证

Python 语法检查;
CPU 复数单精度 4^4 ;
Wilson/Clover + Gauge setup;
staggered transfer $1 \rightarrow 2$;
materialized 与 matrix-free;
QCU blocked transfer/33 点槽位;
QCU 32 方向 coarse halo 与 2/4 rank Clover
MG;

尚未验证

QUDA 原生 GPU kernel 的逐项性能等价;
QCU/QUDA 完整 MG 的 setup 与性能对照;
QUDA 的完整 CG/CA-CG/Krylov null setup;
大格/更多 MPI 拓扑、混合精度和生产收敛曲线;
NVSHMEM 与其他生产级通信路径。

reference 不变量 $\rightarrow$ 批量 UV/VUV kernel $\rightarrow$ 确证 MPI halo $\rightarrow$ 确证 多拓扑 full MG $\rightarrow$ 大格/性能对照

来源：入口：solver/_quda_multigrid.py:986-1104,1237-1347; QUDA 参考：coarse_op.cuh, multigrid.cpp。

QCU ABI 桥接: P/R 与完整 33 点粗算子已逐项对齐

transfer 内存布局

$V_{\text{Python}} : [s, c, q, j, X, Y, Z, T];$
 $V_{\text{QCU}} : [E, e, X_c, b_x, Y_c, b_y, Z_c, b_z, T_c, b_t],$ 其中
 $E = N_{\text{coarse spin}} N_{\text{vec}}, e = N_{\text{fine spin}} N_{\text{fine color}}.$
`QudaTransfer.to_qcu_blocked()` 只重排轴并拆分
`aggregate`, 不改变 P 的数值; 共轭仍只出现在 $R = P^\dagger$ 。

统一导出

`QudaMultigrid.qcu_transition_assets()` 按
 $D_\ell \rightarrow D_{\ell+1}$ 配对返回 blocked null vectors、
`sitting/nearest/diagonal stencil` 及几何元数据。

来源: 本页验证 transfer/stencil ABI; 实现: `_quda_multigrid.py`、`cpp/cuda/qcu/include/lattice_multigrid.h`; 测试: `test_quda_transfer_cuda.py`; kernel: `cpp/cuda/qcu/src/multigrid.cu`。

33 点槽位

`sit` (0, 0, 0, 0)
`hop_nn` $\pm \hat{\mu}, \mu = 0, 1, 2, 3$
`hop_diag` $\pm \hat{\mu} \pm \hat{\nu}$, 顺序 `xy, xz, xt, yz, yt, zt`
`extent = 2` 时重复邻居的系数在两个符号槽位均分;
`extent = 1` 时折入 `sit`。`strict=True` 拒绝超出 33 点支撑的非零 block。

直接证据

CPU QCU 布局/槽位/边界: **PASS** 21/21;
QCU CUDA transfer + wide coarse: **PASS** 1/1, 误差 $< 2 \times 10^{-5}$;
批量 stencil builder 消费 blocked V : 9.1×10^{-8} 。

QCU MPI 多拓扑 full MG 已通过，真残差保持在 4.1×10^{-7}

完整 Clover MG 的空间/时间分解

MPI grid	rank	local fine	local coarse	$\ Dx - b\ /\ b\ $
[1, 1, 1, 2]	2	[8, 8, 8, 8]	[4, 4, 4, 2]	4.136×10^{-7}
[2, 1, 1, 1]	2	[4, 8, 8, 16]	[2, 4, 4, 4]	4.131×10^{-7}
[2, 1, 1, 2]	4	[4, 8, 8, 8]	[2, 4, 4, 2]	4.168×10^{-7}

三种拓扑均在第 71 次收敛判定通过；局部奇偶压缩、粗层方向通信和全算子真残差同时闭环。

来源：命令：pyqcu/testing/qcu/strict/quda_comparison/coarse_mpi_equiv.py、coarse_mpi_smoke.py、run_qcu_mg_mpi.py；实现：
cpp/cuda/qcu/include/lattice_multigrid.h。

通信边界回归

确定性 33 点周期参考：

c64 全局 L2 相对误差 5.60×10^{-7} ；

c128 全局 L2 相对误差 1.03×10^{-15} ；

host-staging 非重叠/重叠：

$\text{rel}_{\max} = 0$ 。

证据含义

这证明 QCU 的 32 方向 coarse halo、局部重建与 Clover Schur 求解可跨空间/时间分解运行；不外推 QUDA 生产 kernel 的性能等价。

阶段结论：Python 参考路径已闭环，生产级 setup 与性能仍是边界

主张

新实现正确表达了 QUDA 的核心层级对象：

$$B \rightarrow V \rightarrow (P, R) \rightarrow RDP \\ \rightarrow (X, Y) \rightarrow \text{Schur/cycle}.$$

置信度

确证 CPU 小格代数、QCU 33 点 coarse halo，以及 2/4 rank Clover MG 真残差；**推断** 与 QUDA 源码控制流/存储语义相符；Python setup 语义已由小格回归直接核验。

来源：源码：QUDA 四个核心文件；实现/测试：_quda_multigrid.py、pyqcu/testing/python/test_quda_multigrid.py、cpp/cuda/qcu/include/lattice_multigrid.h。

剩余最大风险

参考路径的精确 materialize 仍是逐点/逐列 Python 实现；当前完整 MG 已覆盖 $[1, 1, 1, 2]$ 、 $[2, 1, 1, 1]$ 、 $[2, 1, 1, 2]$ 三种小格拓扑。compact Schur setup、QCU 生产 setup、QUDA kernel 性能与大格 scaling 尚未形成证据。

旧实现定位

旧类继续作为历史基线；新类是补全 QUDA 语义的并行实现，不做无声替换。

下一步可验收产物

1. compact parity setup 的伴随与局部块逆；
2. QCU 生产 setup、MPI full residual 与大格 benchmark；
3. setup 成本、迭代数与性能对照表。

需要决策

本轮已将 reference 不变量与 QCU 多拓扑 33 点 MPI 结果固定为迁移的 golden contract；后续若要声明硬件收益，还需批准大格 benchmark 资源。

null-vector setup: QUDA 两种语义与五类 solver 已可验证

输入: $(D_\ell, B_\ell, \text{setup_tol}, \text{setup_max_iter})$

NULL: $D\delta = DB$, 更新 $B \leftarrow B - \delta$

TEST: $DB_{\text{new}} = B$, 零初值求解

solver: BiCGStab/FGMRES 作用于 D

solver: CG/CA-CG 默认作用于 $D^\dagger D$

每轮: 可选全局正交化 $\rightarrow$ setup solver

$\rightarrow$ 后正交化 $\rightarrow$ aggregate-local CGS

输出: $B_\ell \rightarrow V_\ell \rightarrow (P_\ell, R_\ell) \rightarrow D_{\ell+1} = R_\ell D_\ell P_\ell$

来源: 实现: `_quda_multigrid.py:1451-1626`。

测试: `test_quda_multigrid.py:304-389`。

命令: `pytest -q pyqcu/testing/python`。

伴随与边界

自定义 D 必须提供 `fine_adjoint`;
Wilson/Clover 自动使用 $D^\dagger = \gamma_5 D \gamma_5$ 。
`setup_operator=schur` 暂未实现
`compact parity` 伴随, 显式报错。

直接证据

4^4 、复数单精度: **PASS** 21/21; test-vector 精确匹配 $D = 2I$ 的 $B/2$; 五类 setup 均有限且通过 transfer; Wilson γ_5 伴随相对内积误差 $< 2 \times 10^{-5}$ 。

附录：源码—对象索引与证据等级

对象	QUDA 证据	PyQCU 证据/状态
geometry / spin map	transfer.cpp:200-257	_quda_multigrid.py; map、Wilson/Clover 与 staggered
block CGS	block_orthogonalize.cuh:149-239	_quda_multigrid.py; aggregate-local Gram
P / R	transfer.cpp:259-328	_quda_multigrid.py; $R = P^\dagger$
coarse X/Y	coarse_op.cuh:1040-1459	_quda_multigrid.py; Galerkin RDP
PC / Yhat	coarse_op_preconditioned.cuh:48-120	_quda_multigrid.py; 方向/存储
parity / cycle	multigrid.cpp:1131-1224	_quda_multigrid.py; Schur + FGMRES
生产/QCU 后端	CUDA field order、ghost、MPI、MMA、mixed precision	QCU 33 点 coarse halo 与 2/4 rank Clover MG 确证; QCU 生产 setup、QUDA kernel 性能与大格 scaling 未验证

来源：证据口径：确证= 命令直测；推断= 源码推断；未验证= 尚未运行或不在范围。表中路径相对 refer/git-rep/quda、pyqcu；日志：logs/v20260829.txt。

附录：本次交付清单

实现： pyqcu/solver/_quda_multigrid.py (新 QUDA 风格参考路径)

测试： pyqcu/testing/python/test_quda_multigrid.py (21 项 CPU 代数/setup/求解器回归)

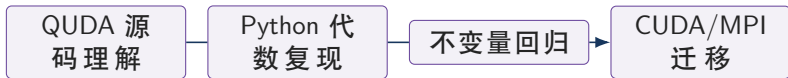
QCU/MPI: pyqcu/testing/qcu/strict/quda_comparison/test_quda_transfer_cuda.py
pyqcu/testing/qcu/strict/quda_comparison/coarse_mpi_equiv.py、pyqcu/testing/qcu/strict/quda_comparison/run_qcu_mg_mpi.py

兼容： pyqcu/solver/__init__.py、pyqcu/cann/__init__.py

稳健性： pyqcu/solver/_gmres.py (zero RHS 与 breakdown)

报告： docs/report_quda_multigrid_reproduction_20260829_3.tex/.pdf

保留： 旧 pyqcu/solver/_multigrid.py 未被替换



生产 setup/性能，未包含

来源：回归：Python 21/21；CUDA 1/1（沿用前一阶段证据）。